Google

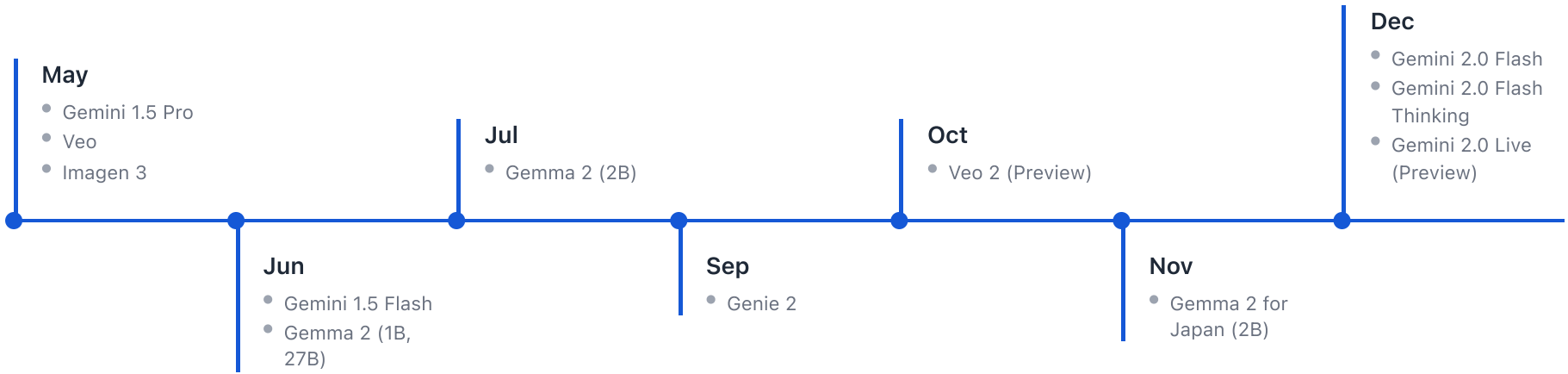
Prompt to Production

Building and deploying a coding agent from scratch with the Interactions API, AI Studio and Antigravity

Ivan Leo , ileo@google.com, ivanleo.com

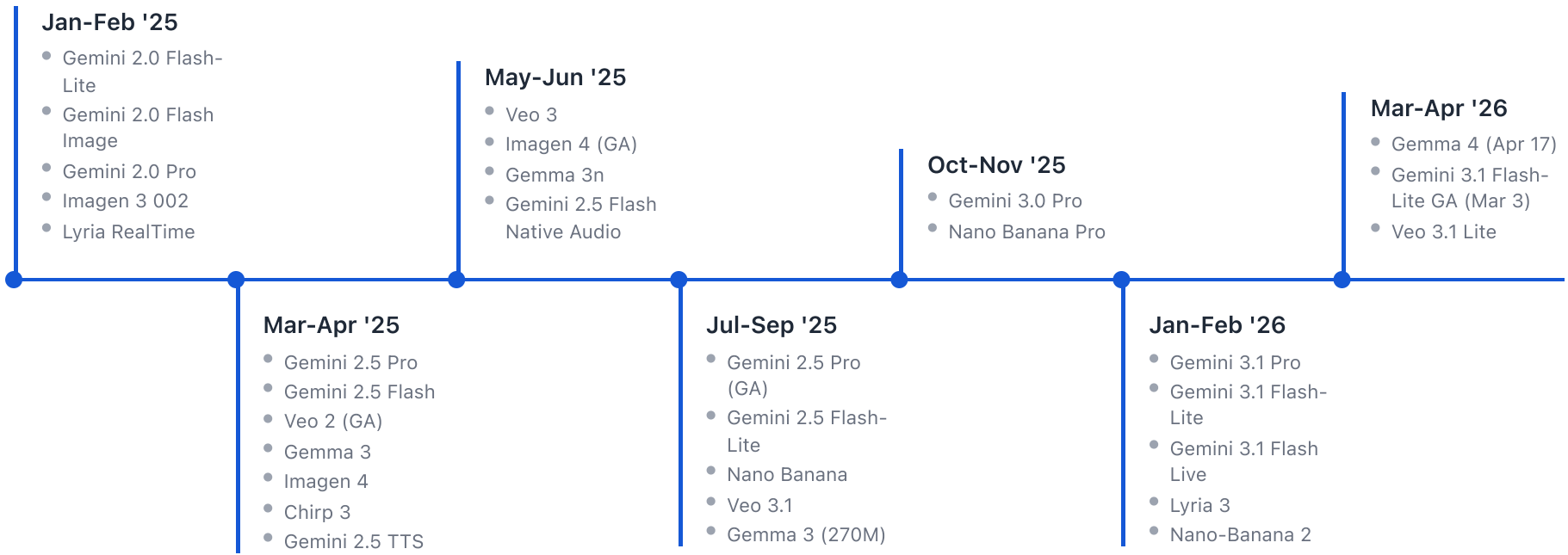
May 2026

Shipping at relentless pace — 2024



Google

...and it's only accelerating — 2025 & 2026



Google

Before we begin...

We'll need a Gemini API key to use the SDK and start building our agent.

- Head over to **aistudio.google.com/app/apikey**
- Click **Create API Key**
- Save it in your project's `.env` file as `GEMINI_API_KEY`

Agenda

01 The Interactions API

02 Building a Coding Agent

03 Questions

Google

01

The Shift Towards Agents

We're moving away from a world where we have simple calls - agents now run without supervision for hours. What comes next?

Google

The Problem

Building agentic workflows today is harder than it should be

- **Manual State Management:** Prone to errors. Breaking the context cache with a single random whitespace can cost significantly more since cached tokens are 10x cheaper.
- **Fragmented Interfaces:** Models and agents (like Deep Research) are behind different endpoints. Divergent APIs and response types make it difficult to chain models.
- **Orchestration sucks:** Switching between different agents to maximize performance is really tough

Google

Generate Content

Our current GenerateContent API makes this process a bit challenging

- Users need to manage state themselves - which can be error-prone and costly
- Deeply nested input/output structures make it hard to reason about the API and know what to send where
- Different APIs for models and agents creates fragmentation - `interactions.create` for deep research vs `generateContent` for our models.

```
# Stateless: re-send history every turn
# Request 1
r1 = client.models.generate_content(
  model="gemini-3-flash-preview",
  contents=[
    {"role": "user", "parts": [
      {"inline_data": {"mime_type": "image/jpeg",
      {"text": "Describe this image"}
    ]}
  ]
)
# Request 2: MUST resend image and history
r2 = client.models.generate_content(
  model="gemini-3-flash-preview",
  contents=[
    {"role": "user", "parts": [
      {"inline_data": {"mime_type": "image/jpeg",
      {"text": "Describe this image"}
    ]},
```

Google

Gemini 3.1 Pro Pricing

Category	Cost (<= 200k tokens)	Cost (> 200k tokens)
Input Tokens	$2.00 / 1M	$4.00 / 1M
Output Tokens	$12.00 / 1M	$18.00 / 1M
Cached Input	$0.20 / 1M	$0.40 / 1M

*Storage price for cached tokens is $4.50 / 1,000,000 tokens per hour.

Google

The Economics of Caching

Turn	Agent Action	Context Size	Uncached Cost	Cached Cost
1	`read_directory("/src")`	500,000 tokens	$2.09	$2.09 (Miss)
2	`run_command("npm test")` (Fails)	505,000 tokens	$2.11	$0.31 (Hit)
3	`view_file("utils.ts")`	510,000 tokens	$2.13	$0.31 (Hit)
4	`replace_file_content(...)`	515,000 tokens	$2.15	$0.31 (Hit)
5	`run_command("npm test")` (Passes)	520,000 tokens	$2.17	$0.32 (Hit)
	Total Workflow Cost		**$10.65**	**$3.34**

Google

The Interactions API

An interaction is a complete turn in a conversation or task

- **Perfect Cache Hits**: An interaction contains the entire history of the agent's history. Say goodbye to expensive cache busting.
- **Interoperability:** Pass state seamlessly between different models - for instance use deep research for heavy lifting, then pass the result to a smaller model for summarization or reformatting.
- **Use Models and Agents:** Both are accessible through the exact same `interactions.create` method.

The Interactions API in Action

```python
from google import genai
client = genai.Client()

# Turn 1: Deep Research agent
i1 = client.interactions.create(
    agent="deep-research-pro-preview-12-2025",
    input="Research AI agents in 2026",
    background=True
)
# ... poll until i1.status == "completed"

# Turn 2: Nano Banana model on same chain
i2 = client.interactions.create(
    model="gemini-3.1-flash-image-preview",
    input="Generate a infographic for the repo
    previous_interaction_id=i1.id
)
print(i2.outputs[-1].text)
```

Google

Flat API Shape

We made the API much easier to reason about

- **No Nested Structures**: A simple, flat set of objects replaces deeply nested historical arrays.
- **Type Parameter**: We use a `type` field to distinguish the type of input and output content blocks.
- **Explicit Status**: Use the `status` of an interaction to clearly determine what action to take next.

RESPONSE SHAPE

```
{
  "status": "requires_action",
  "outputs": [
    {
      "type": "thought",
      "signature": "abc123..."
    },
    {
      "type": "function_call",
      "id": "gth23981",
      "name": "get_weather",
      "arguments": {"location": "Boston"}
    }
  ]
}
```

POST /v1beta/interactions

Google

Content Block Types

Type	Example	Description
Text	`{"type": "text", "text": "Hello"}`	Standard text input or output.
Image	`{"type": "image", "uri": "..."}`	Image resources for multimodal tasks.
Function Call	`{"type": "function_call", ...}`	A request from the model to run a tool.
Function Response	`{"type": "function_response", ...}`	The result of a tool execution.
Thought	`{"type": "thought", ...}`	The model's internal reasoning trace.

Built-in Tools

Tool	Definition	Description
Google Search	`tools=[{"type": "google_search"}]`	Ground responses with real-time web results.
Code Execution	`tools=[{"type": "code_execution"}]`	Execute Python code server-side in a sandbox.
URL Context	`tools=[{"type": "url_context"}]`	Fetch and read full web page content.
Computer Use	`tools=[{"type": "computer_use"}]`	Browser automation via the API.
File Search	`tools=[{"type": "file_search", ...}]`	Search uploaded files in RAG stores.
Remote MCP	`tools=[{"type": "mcp_server", ...}]`	Connect external tools via MCP protocol.

Google

02

Creating our Agent

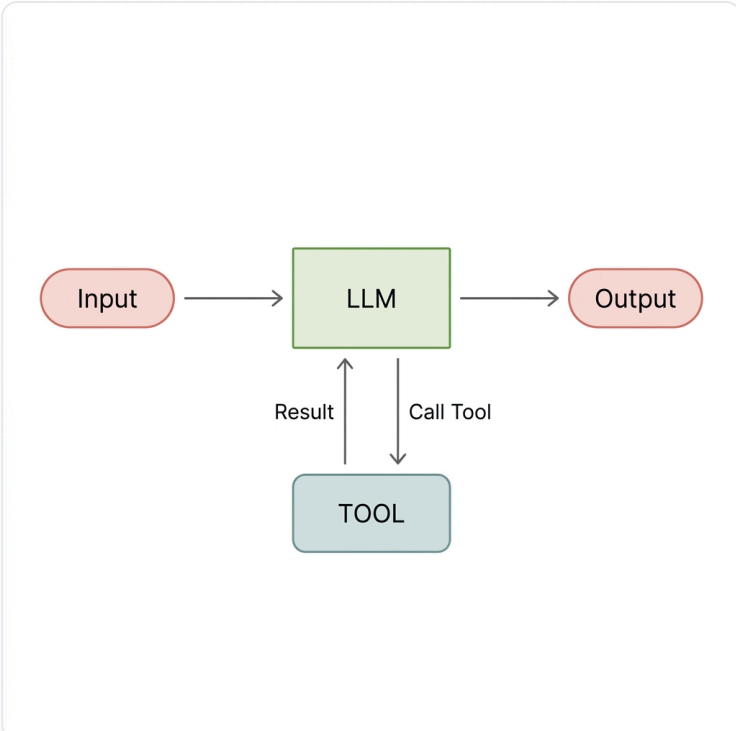
Building a coding agent from scratch with the Interactions API.

Google

What is an Agent?

Now we have an agent

- **The Model (Brain):** The reasoning engine that plans and decides when to use tools.
- **Tools (Hands & Eyes):** Functions that interact with the environment (e.g., Bash, Read, Edit).
- **Context (Workspace):** The accumulated information from previous interactions.
- **The Loop (Life):** A continuous cycle of `Observe → Think → Act → Repeat`.



Google

Building our agent

Let’s get our hands dirty

Google

gemini-interactions-api

Here is a skill to install in order to start using the interactions API.

Install with skills.sh

```
npx skills add google-gemini/gemini-skills --skill gemini-interactions-api --global
```

Install with Context7

```
npx ctx7 skills install /google-gemini/gemini-skills gemini-interactions-api
```

Tool calling basics

Imagine we want our LLM to read a file or search for a restaurant.

- We need a way to know when the model needs to take an action
- If we get the model to generate text like "COMMAND: read_file...", we then need to parse it and execute it
- This is incredibly brittle! If the model formats the text slightly differently, our parser breaks.

This is what we want to avoid

```
import re

response = """I think I should read the file.
COMMAND: read_file('main.py')"""

# Good luck maintaining this across models...
match = re.search(r"COMMAND: (.*)\('(.*)'\)",
if match:
    tool = match.group(1)
    arg = match.group(2)

response = """I think I should edit the file.
COMMAND: process_file('main.py')"""
```

Google

The Solution: Tool Calling

Tool calling is a native mechanism that provides a **guaranteed JSON contract**.

- **Structure over Text:** Instead of guessing intent from text, the model returns a structured data payload.
- **Strict Schema:** You provide the schema upfront (e.g., "I have a tool called `read_file` that takes a `path`").
- **Reliable Execution:** You can confidently map the JSON output directly to your code's functions.

STRUCTURED JSON (RELIABLE)

```
{
  "name": "read_file",
  "arguments": {
    "path": "main.py"
  }
}
```

Google

Invoking our first Chat

The Interactions API makes it extremely simple to chat with a model.

- **Unified API:** Use the exact same endpoint for text generation and tool calling.
- **Automatic State:** The API handles the context caching and message history for you.

BASIC INTERACTION

```
const interaction = await client.interactions.
  model: "gemini-3-flash-preview",
  input: "Hello! Who are you?"
});

console.log(interaction.outputs.at(-1).text);
```

Google

Tool Use

We can define a JSON schema to guarantee the output format when a model needs to interact with the environment.

- **Schema Definition:** Provide the type, name, description, and properties.
- **Attaching Tools:** Simply pass the list of tools to the `interactions.create` call.
- **Model Decides:** The model will choose whether to output a text response or a `function_call`.

DEFINING TOOLS

```
const tools = [{
  type: "function",
  name: "get_weather",
  description: "Get current weather",
  parameters: {
    type: "object",
    properties: {
      location: { type: "string" }
    }
  }
}];

const interaction = await client.interactions.
  model: "gemini-3-flash-preview",
  input: "What's the weather in Paris?",
  tools
});
```

Google

Creating the loop

An agent is just a `while` loop wrapped around an LLM.

- **Check Status:** Keep looping as long as `interaction.status == "requires_action"`.
- **Execute Tools:** Iterate through the function calls, run your local code, and collect results.
- **Pass Back:** Provide the tool results as the next `input`, and reference the `previous_interaction_id` to maintain context.

THE AGENT LOOP

```
// Initial call with tools
let interaction = await client.interactions.c
    model: "gemini-3-flash-preview",
    input: "Read example.txt and summarize it'
    tools
});

// Agent loop: keep going while model needs t
while (interaction.status === "requires_actio
    const results = [];
    for (const output of interaction.outputs)
        if (output.type === "function_call") {
            const result = await execute(outpu
            results.push({
                type: "function_result",
                name: output.name,
                call_id: output.id,
                result
```

Google

Securing the Workspace

Building coding agents means giving a model the ability to execute code and bash commands. Doing this locally is dangerous.

- **The Danger:** A hallucination could result in `rm -rf /` on your local laptop.
- **The Solution:** We use **Daytona Sandboxes**.
- **Ephemeral & Isolated:** Every agent interaction spins up a secure, isolated cloud environment.
- **Seamless Integration:** Execute bash commands inside the sandbox exactly as if they were local.

DAYTONA INTEGRATION

```
// 1. Initialize Daytona Context
const daytona = new Daytona();
const sandbox = await daytona.create({
  language: 'typescript'
});

// 2. Execute safely in the sandbox
const response = await sandbox.process.execute
  args.command
);
console.log(response.result);
```

Google

Agent Observability

Once an agent is running autonomously, how do you know what it actually did?

- **The Black Box:** Agents might take 20 turns before giving you a final answer. If something breaks, debugging is a nightmare.
- **OpenTelemetry & Logfire:** By wrapping our API calls in OTEL spans, we get deep visibility.
- **Track Everything:** See exact token usage, the full prompt/response payloads, and exactly which tools the model decided to call and when.

LOGFIRE TRACING

```
const response = await logfire.span(
  'Gemini API Call',
  {
    model: "gemini-3-flash",
    input: currentInput
  },
  async () => {
    return await client.interactions.create({
      model: "gemini-3-flash",
      input: currentInput,
      previous_interaction_id: prevId,
      tools: activeTools
    });
  }
);
```

Google

Time Travel Debugging

Testing a new prompt or system persona usually requires re-running the entire 5-minute conversation loop.

- **The Power of `previous_interaction_id`:** Because the API maintains state on the server, we can extract the exact interaction ID from a Logfire trace.
- **Perfect Replay:** We can pull the state right before the final response, inject a localized system reminder, and replay only the final generation.
- **Rapid Iteration:** Test new personas (e.g., formal vs casual) in seconds without re-executing any tools.

INJECTING REMINDERS

```
// 1. Grab ID from your observability trace
const traceId = "v1_ChdJSW...";

// 2. Inject your reminder
inputPayload.push({
  type: "text",
  text: "SYSTEM REMINDER: Please reply like a
});

// 3. Replay from exact state!
const response = await client.interactions.cre
    input: inputPayload,
    previous_interaction_id: traceId,
});
```

Google

03

Questions?

What can we help unblock you with today!

Google